INFORME DE JAVASCRIPT

Estudiante:	Soledad Pinilla
Proyecto:	L&S Gestión Comercial – Librería y Fotocopiadora
Fecha:	7 de noviembre de 2026
Repositorio GitHub:	https://github.com/soledadpinilla949-cloud/laboratorio-2026
Carpeta del proyecto:	LS_GESTION

- Qué es el DOM
  - ¿Qué es y por qué se lo llama "árbol"?
  - ¿Qué es un nodo y qué es un elemento?
  - ¿Qué relación hay entre el HTML y el DOM?
- Selección de elementos
  - querySelector y querySelectorAll: ¿qué devuelve cada uno?
  - ¿Qué pasa si querySelector no encuentra nada?
  - ¿Cómo se recorre el resultado de querySelectorAll?
- Modificar el DOM
  - TextContent vs innerHTML: ¿en qué se diferencian?
  - classList: add, remove, toggle y contains
  - createElement, append y remove
- Template strings
  - Comillas invertidas y ${}
  - ¿Cómo se arma el HTML de una card?
  - ¿Para qué sirve .join("") junto con map?
- Eventos
  - ¿Qué es un evento? Dar 4 ejemplos (click, input, submit, change)
  - addEventListener: ¿qué recibe y qué hace?
  - El objeto event, y la diferencia entre e.target y e.currentTarget
- Delegación de eventos
  - ¿Qué es y qué problema resuelve?
  - ¿Qué hace e.target.closest(...)?
  - ¿Por qué es mejor un solo listener en el contenedor que uno por botón?

- Atributos data-* y dataset
  - ¿Qué son y cómo se leen?
  - ¿Por qué hay que convertir con Number(...) un data-id?
- preventDefault()
  - ¿Qué evita? Ejemplo con un formulario y con un enlace
  - ¿Qué pasa si te olvidás de usarlo en un formulario?

1. ¿Qué es el DOM?

El DOM significa **Document Object Model** o Modelo de Objetos del Documento. Es una representación del documento HTML que el navegador crea para poder trabajar con su contenido desde javascript.

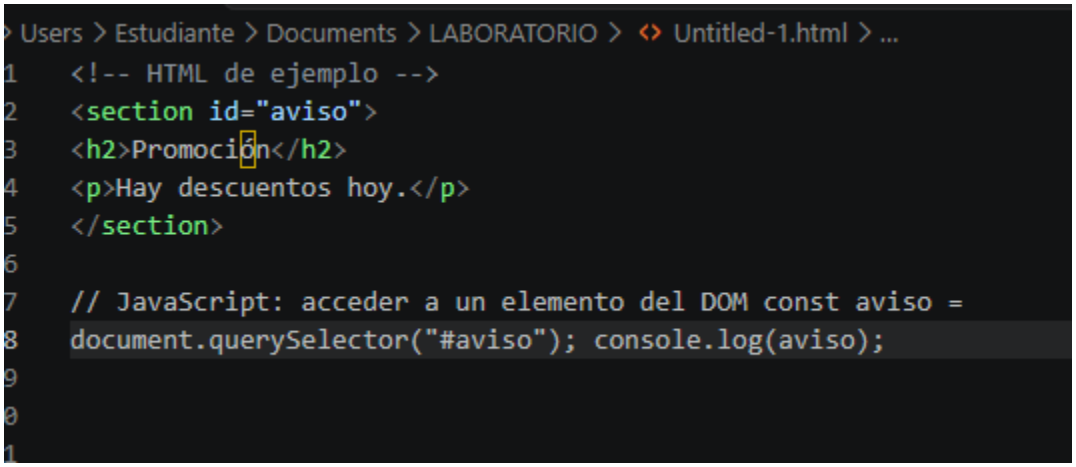
Se lo llama “árbol” porque organiza los elementos en una estructura jerárquica: hay un documento principal, elementos padres, elementos hijos y otros elementos que pueden estar al mismo nivel.

Un **nodo** es cada parte de esa estructura; pueden ser elementos, textos, comentarios, entre otros. Un **elemento** es un tipo de nodo que representa una etiqueta HTML, como p, div o button.

El HTML es el código que escribimos para definir la estructura inicial de la página. El DOM es la representación que construye el navegador a partir de ese HTML.

Gracias al DOM, javascript puede consultar, modificar, agregar o quitar contenido sin tener que volver a escribir todo el archivo HTML.

Ejemplo propio en código



2. Selección de elementos

Para trabajar con una parte de la página primero necesitamos seleccionarla. Queryselector() busca usando un selector CSS y devuelve el **primer elemento** que coincide con ese selector.

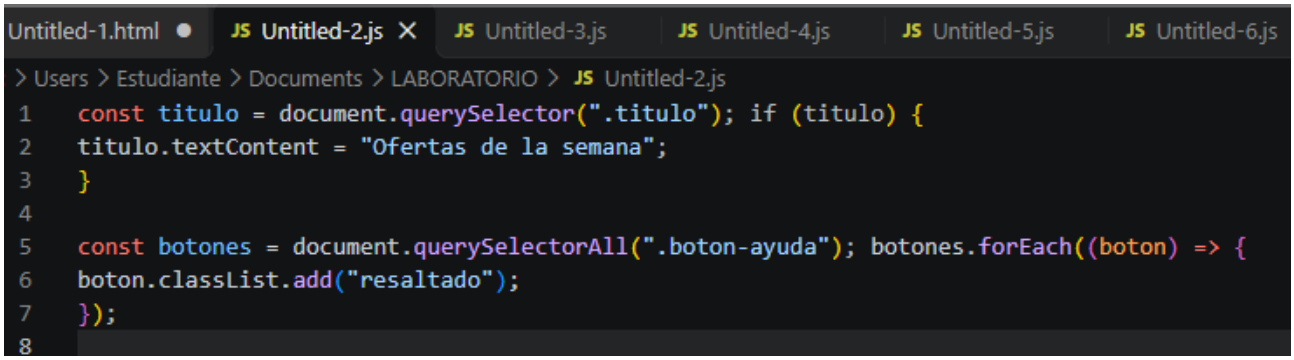
En cambio, queryselectorall() devuelve todos los elementos que coinciden, dentro de una colección llamada **nodelist**. Si no encuentra ninguno, devuelve una nodelist vacía.

Si queryselector() no encuentra coincidencias, devuelve null. Por eso, antes de usar ese resultado, conviene comprobar que exista.

El resultado de queryselectorall() se puede recorrer con foreach(), por ejemplo para cambiar el estilo de varios elementos.

La diferencia principal es que uno selecciona un solo elemento (el primero) y el otro reúne todos los elementos coincidentes.

Ejemplo propio en código



3. Modificar el DOM

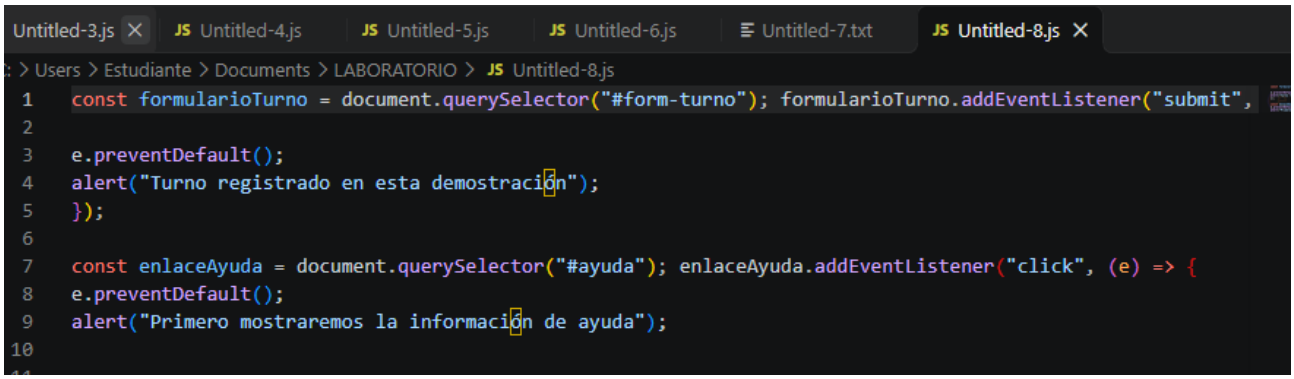
Javascript puede cambiar el contenido y la apariencia de una página mediante el DOM. Textcontent permite leer o cambiar texto plano; no interpreta etiquetas HTML.

Innerhtml permite leer o establecer contenido HTML, por lo que interpreta las etiquetas que se escriben dentro. Hay que usarlo con cuidado si el contenido viene de usuarios o de una fuente no confiable.

Classlist sirve para trabajar con las clases CSS: add() agrega una clase, remove() la quita, toggle() la activa o desactiva y contains() comprueba si está presente.

Con createelement() se crea un elemento nuevo. Después, append() lo agrega dentro de otro elemento y remove() elimina un elemento del documento.

Ejemplo propio en código



4. Template strings

Las template strings son cadenas de texto escritas con comillas invertidas: ` `. A diferencia de las comillas simples o dobles, permiten insertar variables o expresiones usando ${}.

Esto resulta práctico para construir fragmentos de HTML desde javascript, porque podemos combinar etiquetas con datos que cambian, como el nombre, el precio o la descripción de un artículo.

Para armar una card, se prepara una plantilla con la estructura HTML y se colocan dentro los valores de cada objeto.

El método map() permite transformar cada elemento de un array en otro valor; por ejemplo, convertir cada objeto en una tarjeta HTML.

Al final, .join("") une todos los textos generados en una sola cadena, sin separadores, para poder insertarla en el contenedor.

Ejemplo propio en código

C: > Users > Estudiante > Documents > LABORATORIO > JS Untitled-4.js

```
1   const servicios = [
2   { nombre: "Anillado", precio: 1800 },
3   { nombre: "Impresión color", precio: 500 }
4   ];
5
6   const tarjetas = servicios.map((servicio) => `
7   <article class="card">
8   <h3>${servicio.nombre}</h3>
9   <p>Precio: $${servicio.precio}</p>
10  </article>
11
12  `).join(""); document.querySelector("#servicios").innerHTML = tarjetas;
13
```

5. Eventos

Un evento es una acción o situación que ocurre en la página y que javascript puede detectar para ejecutar una respuesta.

Algunos ejemplos son **click** (cuando se hace clic), **input** (cuando cambia lo que se escribe en un campo), **submit** (cuando se envía un formulario) y **change** (cuando cambia el valor de un control, por ejemplo una lista desplegable).

Addeventlistener() recibe el nombre del evento y una función que se ejecutará cuando ocurra. También puede recibir opciones adicionales.

El objeto event contiene información sobre el evento. E.target es el elemento donde se originó la acción; e.currenttarget es el elemento que tiene registrado el listener que se está

ejecutando.

Estos conceptos permiten responder a las acciones de la persona que usa la página.

Ejemplo propio en código



6. Delegación de eventos

La delegación de eventos consiste en colocar un solo listener en un elemento contenedor para atender eventos que ocurren en elementos hijos.

Es útil cuando hay muchos botones o cuando los elementos se crean dinámicamente después de cargar la página. Así no hace falta registrar un listener nuevo cada vez que aparece un botón.

Cuando ocurre el evento, e.target identifica el elemento exacto donde se hizo clic. El método closest() busca ese elemento o uno de sus padres que coincida con el selector indicado.

Por ejemplo, e.target.closest("[data-accion]") permite encontrar el botón con ese atributo aunque el clic haya ocurrido sobre un ícono o un texto que está dentro del botón.

Es más ordenado y eficiente tener un listener en el contenedor que uno por cada botón, especialmente cuando hay muchos elementos.

Ejemplo propio en código



7. Atributos data-* y dataset

Los atributos data-* son atributos HTML personalizados que permiten guardar información adicional en un elemento. Por ejemplo, se puede guardar el identificador de un artículo con data-id="25".

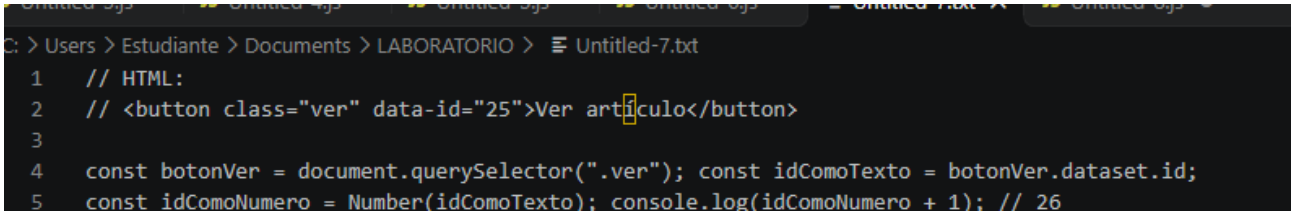
Desde javascript se leen mediante la propiedad dataset. En el ejemplo anterior, se accede con elemento.dataset.id.

Los valores que vienen de los atributos HTML se leen como texto (string), incluso cuando parecen números.

Por eso, si necesitamos realizar una operación matemática o comparar el identificador como número, debemos convertirlo con Number().

La conversión evita comportamientos inesperados: por ejemplo, sumar el texto "2" y el número 1 puede producir "21" en lugar de 3.

Ejemplo



8. Preventdefault()

Preventdefault() es un método del objeto evento que evita que el navegador realice la acción predeterminada asociada a ese evento.

En un formulario, normalmente el navegador intenta enviarlo y puede recargar o navegar a otra página. Si queremos validar los datos o procesarlos con javascript, usamos e.preventdefault() dentro del evento submit.

En un enlace, evita que el navegador siga automáticamente la dirección indicada en href. Esto puede servir, por ejemplo, para mostrar una confirmación antes de continuar.

Si nos olvidamos de usarlo en un formulario que debía manejar javascript, el navegador puede enviar el formulario de la manera tradicional y recargar la página, perdiendo cambios que todavía no se guardaron.

No cancela cualquier acción: solo evita el comportamiento predeterminado del evento que se está procesando.

Ejemplo

C: > Users > Estudiante > Documents > LABORATORIO > JS Untitled-8.js

```
1   const formularioTurno = document.querySelector("#form-turno"); formularioTurno.addEventListener("submit",
2
3   e.preventDefault();
4   alert("Turno registrado en esta demostración");
5   });
6
7   const enlaceAyuda = document.querySelector("#ayuda"); enlaceAyuda.addEventListener("click", (e) => {
8   e.preventDefault();
9   alert("Primero mostraremos la información de ayuda");
10
11
```